



华中科技大学 2024~2025 第二学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试时间 2025-5-13 晚上 考试时长 150 分钟

院 (系) _____ 专业班级 _____

学 号 _____ 姓 名 _____

题号	一	二	三	四	五			总分	总分人	核对人
分值	20	20	30	20	10			100		
得分										

分 数	
评卷人	

一、(20 分) 计算机工作的基本原理。

1、正常控制流下的程序执行 (10 分, 每空 1 分)

① 冯·诺依曼体系结构的基本思想是存储程序和自动执行。程序计数器 的自动变化是实现程序自动执行的核心机制之一。以下是 Intel CPU 中的 rip 自动变化的方法。在加载运行一个程序时, 操作系统和 CPU 会协同工作, 将可执行文件中记载的 入口地址 送入 rip 中。设 rip 为 0x08001144, 该处指令为“c7 45 f4 05 00 00 00 movl \$0x5,-0xc(%rbp)”, 在取出该指令并进行译码后, rip = 0x 800114b。在 0x0800114f 处, 有指令 “75 09 jne 0x800115a”, 若执行该指令前, ZF=0, 指令执行后 rip = 0x 800115a; 若 ZF=1, 指令执行后 rip = 0x 8001151。

在 0x08001158 处, 有反汇编语句“eb xx jmp 0x8001161”, 这里机器码(1 个字节) xx = 07。

在执行 CALL 指令时, CPU 会将 rip 压入堆栈栈顶, 同时会将 子程序入口地址 送入 rip 中。在子程序返回时, 执行 ret 指令, 会从 栈顶弹出 8 个字节 送入 rip 中。

2、异常控制流下的 rip 的变化 (10 分, 每空 1 分):

在执行一条指令后, CPU 检测到有按键操作, 此时会发生 中断。CPU 会依次保存标志寄存器和 rip, 同时根据中断号从 中断矢量表 中找到 中断处理程序的入口地址 送给 rip。中断处理程序保护恢复通用寄存器。iret 指令先将 栈顶 8 个字节弹出送 RIP, 再 弹出标志寄存器, 从而恢复被打断程序的继续执行。若 CPU 在执行除零指令或者指令中访问的地址超出了本程序的空间范围, 此时会产生 异常。CPU 执行异常处理程序后, 若继续执行引起异常的指令, 则称之为 故障; 若执行引起异常的指令的下一条指令, 则称之为 陷阱, 其代表性的事件有 软中断或者调试跟踪。

解答内容不得超过装订线

分 数	
评卷人	

二、(20 分) 数据的存储和地址表达形式

1. 数据的存储形式 (10 分)

设有如下程序段，在运行时观察了各变量的地址如下

```
int x=10;    // &x = 0x08004010

int y=-10;   // &y = 0x8004014

char str[10]="20250513";    // &str[0] = 0x8004018

char a[4]={255,-1,257,-257}; // &a[0] = 0x8004022

float f=20.5; // &f = 0x8004028

short u=30;   // &u = 0x800402c
```

以 16 进制的形式填写程序用到的内存单元内容(无关的字节中的内容填写 XX, 每行 2 分)。

0x08004010: 0A 00 00 00 F6 FF FF FF
 0x08004018: 32 30 32 35 30 35 31 33
 0x08004020: 00 XX FF FF 01 FF XX XX
 0x08004028: 00 00 a4 41 1E 00 XX XX。

(int)(str+2) 的值是 0x35303532

(char)(&x+1) 的值是 0xF6

提示: '0' 的 ASCII 为 0x30; 浮点数 float 的编码规则: float 用 32 位表示, 最高位(第 31 位)为符号位, 23-30 位为用移码表示的指数(移码值为 0x7F), 0-22 位为尾数位。

2. 根据调试一个 C 程序时看到的如下信息填空。(10 分, 每空 1 分)

```
int a[5];
```

```
int i=3;
```

```
movl $0x3, -0x44(%rbp) // rbp = 0x7ffffffe040
```

此时, i 的地址是 0x7ffffffdffc, i 存放在堆栈段中。i 的地址表达形式是-0x44(%rbp), 该表达形式是在生成执行程序时就固定不变了。但是 i 的具体地址值在不同的执行时刻, 因 rbp 的变化而变化。

```
int *p=&a[0];
```

```
lea -0x20(%rbp), %rax ; 执行后, rax = 0x7ffffffe020
```

```
mov %rax, -0x38(%rbp) ; p 的地址是 0x7ffffffe008
```

```
a[0]=1;
```

```

movl    $0x1, -0x20(%rbp)
*(a+1)= 2 ;
movl    $0x2, -0x1c(%rbp)
*(p+2)=3;
mov     -0x38(%rbp), %rax
add     $0x08, %rax
movl    $0x3, (%rax)
a[i]=5;
mov     -0x44(%rbp), %eax
cltq    ; 将 rax 的高 32 位清零
movl    $0x5, -0x20(%rbp,%rax, 4 )

```

分 数	
评卷人	

三、(30 分) C 程序与机器语言的对应

1、数据传送、算术运算、按位运算、移位运算 (10 分, 每小题 1 分)

(写汇编语句时, 要求有指令助记符、操作数和操作数地址, 用变量符号名表示操作数地址)

设有 `int x=0x12345678;`

`unsigned short u=0xffff; short v=0xffff;`

分别执行如下语句 (每条语句执行前, x 都是 0x12345678)

- ① `x=u;` 数据扩展传送指令的指令助记符是 movzwl, 执行后, `x=` 0x0000ffff。
- ② `x=v;` 数据扩展传送指令的指令助记符是 movswl, 执行后, `x=` 0xffffffff。
- ③ `u=u>0;` `u>0` 是 (是、否) 成立; 执行后 `u=` 1。
- ④ `v=v>0;` `v>0` 否 (是、否) 成立; 执行后 `v=` 0。
- ⑤ `x = x>>2` 对应的汇编语句是 sar \$2, x, 执行后, `x=0x` 048d159e。
- ⑥ `x = x<<1` 对应的汇编语句是 shl \$1, x, 执行后, `x=0x` 2468acf0。
- ⑦ `x=x&0x0f` 对应的汇编语句是 and \$0x0f, x, 执行后, `x = 0x` 00000008。
- ⑧ `x = -x` 对应的汇编语句是 neg x, 执行后, `x = 0x` edcba988。
- ⑨ `x = ~x` 对应的汇编语句是 not x, 执行后, `x = 0x` edcba987。
- ⑩ `x=x+10` 对应的汇编语句是 addl \$0x0a, x, 执行后, `x = 0x` 12345682。

2、分支转移程序（10 分，每空 1 分）

① if 语句与汇编语句的对应（5 分，每空 1 分）

（L1、L2、L3、L4 皆为标号，代表机器指令的地址）

```
int x=-1;    int y=1;

    movl    $0xffffffff, -0xc(%rbp)

    movl    $0x1, -0x8(%rbp)

int flag;

if (x>y)    flag=1;

    mov     -0xc(%rbp), %eax

    cmp     -0x8(%rbp), %eax

L1:  jle    L3

    movl    $0x1, -0x4(%rbp)

L2:  jmp    L4

else    flag=0;

L3:  movl    $0, -0x4(%rbp)

L4:  .....
```

若将变量 x、y 定义为 unsigned int 类型，L1 处的汇编语句应为 jbe L3。

若漏写了 L2 处的语句，与该 if 语句功能等价的 C 语句是 flag=0。

② switch 语句与汇编语句的对应（5 分，每空 1 分）

```
int x=3;

    movl    $0x3, -0x8(%rbp)

int y;

switch (x)

    cmpl    $0x1, -0x8(%rbp)

    je      L1

    cmpl    $0x2, -0x8(%rbp)

    je      L3

    jmp     L5

{
```

case 1: y=1; break;

L1: movl \$0x1, -0x4(%rbp)

L2: jmp L6

case 2: y=2; break;

L3: movl \$0x2, -0x4(%rbp)

L4: jmp L6

default: y=3;

L5: movl \$0x3, -0x4(%rbp)

}

L6:

若将 default 分支移到 case 1 之前, switch 语句执行后, y= 1。

若要使 default 语句放在 case1 之前与放在最后的功能等价, default 语句应修改为:

default: y=3 ; break ;。

3、函数调用与汇编语句的对应 (10 分)

有如下主程序:

int a = 1;

0x000000000800116c <main+8>: movl \$0x1, -0x4(%rbp)

int b = 2;

0x0000000008001173 <main+15>: movl \$0x2, -0x8(%rbp)

int sum = 0;

0x000000000800117a <main+22>: movl \$0x0, -0xc(%rbp)

sum = fadd(a, b);

0x0000000008001181 <main+29>: mov -0x8(%rbp), %edx

0x0000000008001184 <main+32>: mov -0x4(%rbp), %eax

0x0000000008001187 <main+35>: mov %edx, %esi

0x0000000008001189 <main+37>: mov %eax, %edi

0x000000000800118b <main+39>: callq 0x8001139 <fadd>

0x0000000008001190 <main+44>: mov %eax, -0xc(%rbp)

int fadd(int x, int y) 函数和反汇编代码如下:

{

0x0000000008001139 <+0>: push %rbp

0x000000000800113a <+1>: mov %rsp, %rbp

0x000000000800113d <+4>: mov %edi, -0x14(%rbp)

0x0000000008001140 <+7>: mov %esi, -0x18(%rbp)

int u, v, w;

u = 2*x ;

```

0x0000000008001143 <+10>:  mov    -0x14(%rbp), %eax
0x0000000008001146 <+13>:  add     %eax, %eax
0x0000000008001148 <+15>:  mov     %eax, -0x4(%rbp)
        v = y + 25;
0x000000000800114b <+18>:  mov    -0x18(%rbp), %eax
0x000000000800114e <+21>:  add     $0x19, %eax
0x0000000008001151 <+24>:  mov     %eax, -0x8(%rbp)
        w = u + v;
0x0000000008001154 <+27>:  mov    -0x4(%rbp), %edx
0x0000000008001157 <+30>:  mov    -0x8(%rbp), %eax
0x000000000800115a <+33>:  add     %edx, %eax
0x000000000800115c <+35>:  mov     %eax, -0xc(%rbp)
        return w;
0x000000000800115f <+38>:  mov    -0xc(%rbp), %eax
}
0x0000000008001162 <+41>:  pop     %rbp
0x0000000008001163 <+42>:  retq

```

刚进入 fadd 时（即 push %rbp 执行前），

rsp = 0x7ffffffe028；

rbp = 0x7ffffffe040

填写执行到 fadd 的 “pop %rbp” 语句时

的堆栈内容。

要求：在图中标明 x, y, u, v, w 的位置；

每行是 4 个字节，其中的内容用 0x...表示

具有随机性的单元内容填为

XXXXXXXX。

y	0x00000002	0x7ffffffe008
x	0x00000001	0x7ffffffe00c
	XXXXXXXXXX	0x7ffffffe010
w	0x0000001d	0x7ffffffe014
v	0x0000001b	0x7ffffffe018
u	0x00000002	0x7ffffffe01c
	0xfffe040	0x7ffffffe020
	0x00007fff	0x7ffffffe024
	0x08001190	0x7ffffffe028
	0x00000000	0x7ffffffe02c

分 数	
评卷人	

四、(20 分) 程序编译与优化问答

1、程序优化（16 分）

① 求二维数组中所有元素的和，在二重循环中，行序优先访问比列序优先访问快，利用了 CPU 的什么特性和二维数据存储的什么特性？（2 分）

答：CPU 访问数据时使用 CPU 中的 cache，若数据不在 cache 中，就要将内存中的数据块调入 cache，有时间开销。二重数组按行序优先的顺序存放，数据块调入 cache 时，把即将要

访问的数据也就调入了 **cache**。按行序访问，**cache** 的命中率高，速度快。

② 将有分支的语句转换成无分支的语句，或者将循环语句转换成无循环的语句，提高程序执行速度，利用了 CPU 的什么特性？为什么会提高速度？（2 分）

答：CPU 对指令进行处理是用指令流水线方式。提高指令流水线的利用率可加快程序执行速度。在发生转移时，已在流水线上进行加工了的“半成品”要作废，使得指令流水线的利用率低。

③ 将一大片空间中的内容置为 0，可以用循环语句一个字节一个字节的置零，也可以使用 **memset** 函数来完成，后者的执行速度更快，原因是什么？（2 分）

答：**mement** 中封装了串操作指令。CPU 中的串操作指令执行速度更快。

④ 写出与 **imul \$2, %eax** 功能等价但执行速度更快的两种机器指令（2 分）

答：
add %eax, %eax
shl \$1, %eax

⑤ 举例说明，编译器还可以做哪些与 CPU 特性无关的优化？（2 分）

答：去掉无作用的废语句。能在编译时，直接计算出结果的表达式或者函数调用，直接用计算出的结果，而不需要对中间过程产生语句，节省了程序运行时的指令数。

⑥ 设有如下程序段及其未优化的反汇编程序段（6 分）。

```
int a[5];
int sum=0, i;
    movl    $0x0, -0x28(%rbp)
for (i=0; i<5; i++)    sum+=a[i];
    movl    $0x0, -0x24(%rbp)
    jmp     L2
L1:  mov     -0x24(%rbp), %eax
    cltq    ; 将 rax 的高 32 位置 0
    mov     -0x20(%rbp,%rax,4), %eax
    add     %eax, -0x28(%rbp)
    addl    $0x1, -0x24(%rbp)
L2:  cmpl    $0x4, -0x24(%rbp)
    jle     L1
```

写出完成等价功能的尽可能优化的汇编程序段，并指出使用的优化策略。

答：优化策略有：循环展开、变量与寄存器绑定，**sum** 与 **eax** 绑定

```
mov a[0], %eax
add a[1], %eax
add a[2], %eax
add a[3], %eax
add a[4], %eax
mov %eax, sum
mov $5, i
```

2、程序的安全性（4分）

① 为什么不应将函数中局部变量的地址返回给调用函数来使用？（2分）

答：局部变量分配在栈中。在一个函数执行完成后，该栈处于“释放”状态，可供下面执行的函数使用。即意味着相应单元中的内容被其他函数改变，而不再是从函数返回时的内容。

② 编译器生成的执行程序中，可采取何种方法检测出缓冲区溢出攻击？（2分）

答：在保存的断点地址之前，加一个金丝雀（哨兵），该值是一个随机值。在函数返回前，检查该值是否发生变化，若变化，则说明有溢出。

分 数	
评卷人	

五、（10分）程序的链接回答

1、设一个程序中，有全局变量定义语句 `int global=10;` （2分）

在可重定位目标文件中的**哪些节**，会生成与该变量定义有关的**哪些信息**？

答：字符串节，存放有 `global` 的 ASCII 串；

符号表节，记录 `global` 定义在数据节，`global` 的地址，符号是已定义状态；

数据节，存放 `global` 的具体值，有 `0A 00 00 00`。

2、在一个函数中有语句 `int temp=global;`（`global` 为前面定义的全局变量），在可重定位目标文件中的**哪些节**，会生成与该语句有关的**哪些信息**？（2分）

答：代码节，有 `temp=global` 对应的机器指令（非优化状态下 2 条指令）。

代码节的重定位节，有需要定位的信息，包括机器指令的什么位置、用什么符号、以何种重定位方式来重定位，还有计算重定位值时需要的附加值。

3、设函数中有语句 `printf("very good\n");` 字符串“very good\n”存放在什么节中？该 `printf` 语句对应的代码中有几处需要重定位？分别是什么位置需要重定位？（2分）

答：只读数据节，存放“very good\n”的 ASCII 串。有 2 个地方要重定，一是字符串的地址，二是 `printf` 函数的地址。

4、简述链接的执行过程，以及在链接时能够检测到的错误。（4分）

答：可以检测到符号重定义错误、符号未定义错误。

链接是将多个目标文件（.o 或 .obj）和库文件合并成一个可执行文件或共享库的过程。其核心目标是解析符号引用、分配内存地址，并最终生成可运行的程序。

将各个可重定位目标文件中代码节、数据节、只读节等分别合并到一个文件中，在合并时，要检查各个符号是否重复定义，若有重复定义报错；同时要记录哪些符号没有定义（只是引用了外部符号），在文件合并后，对未定义的符号要判断是否出现在链接库中，若还存在未定义符号，也要报错。

根据合并后的文件，确定符号的地址。根据符号地址以及重定位节信息，对程序中需要重定位的地方进行修改，完成重定位工作。